Table de données

La notion de table est apparue dans les années 1970 chez IBM avec l'algèbre relationnelle. Les tables de données, stockées dans des bases de données relationnelles deviennent rapidement le moyen le plus rependu de stocker et traiter de l'information structurée.

Cours

Une table de données est un ensemble d'éléments organisés sous forme d'une table :

- où chaque ligne correspond à un élément (appelé aussi une entité ou une entrée)
- où chaque colonne correspond à une catégorie d'information de même type, appelé un champs ou attribut (une colonne peut stocker des numéros de téléphone, une autre des noms...)

Exemple : une liste d'information sur les pays utilisée au chapitre 'Types construits':

Pays	Capitale	Population (ml)
France	Paris	68
Allemagne	Berlin	82
Italie	Rome	60

Cours

Les titres des colonnes (Pays, Capitale, ...) sont appelés les **descripteurs** de la table.

De nombreux formats de données en table sont disponibles : txt, csv, json, excel, etc.

Le format csv

Cours

Le format csv (*Comma-Separated Values*), est un format texte représentant des données en table séparées par des virgules (comme son nom l'indique) ou d'autres séparateurs, par exemple point-virgule (« `;` »), tabulation (« `\t` »), etc.

Chaque ligne du texte correspond à une ligne du tableau et les virgules correspondent aux séparations entre les colonnes. Un fichier csv est un fichier texte mais on utilise l'extension .csv pour indiquer la nature de ses données.

Exemple : Reprenons la table des pays, en fichier csv on écrit dans un fichier texte.

```
Pays,Capitale,Population (ml)
France,Paris,68
Allemagne,Berlin,82
Italie,Rome,60
```

On peut utiliser des logiciels comme Microsoft Excel[1] ou OpenOffice Calc permettent de visualiser un fichier csv directement. D'autres formats de données structurées existent, par exemple JSO N , XML, etc.

Ecrire une table de données dans un fichier csv

Un fichier csv est un simple fichier txt, on écrire la table des pays dans un fichier csv de la même façon que dans un fichier texte en séparant les données par une virgule (par exemple) et les lignes par `\n` :

```
f = open("pays.csv", "w")              # si pays.csv n'existe pas, il est créé
f.write('Pays,Capitale,Population (ml)\n')
f.write('France,Paris,68\n')
f.write('Allemagne,Berlin,82\n')
f.write('Italie,Rome,60\n')
f.close()
```

Lire une table de données depuis un fichier csv

On peut lire les données depuis un fichier csv (ou un autre type de fichier tabulé) et enregistrer les données dans un tableau de tableaux en le lisant ligne par ligne :

f = open()

```
f = open("pays.csv", "r")
pays = []
for li in f:
    pays.append(li.split(','))
f.close()
```

with open as f:

```
py
with open("pays.csv", "r") as f:
    pays = []
    for li in f:
        pays.append(li.split(','))
```

ou même par compréhension :

f = open()

```
f = open("pays.csv", "r")
pays = [li.split(',') for li in f]
f.close()
```

with open as f:

```
py
with open("pays.csv", "r") as f:
    pays = [li.split(',') for li in f]
```

On obtient le tableau de tableaux suivant:

```
>>> pays
[['Pays', 'Capitale', 'Population (ml)\n'],
 ['France', 'Paris', '68\n'],
 ['Allemagne', 'Berlin', '82\n'],
 ['Italie', 'Rome', '60\n']]
```

On peut utiliser `li[ :-1]` pour supprimer les retours à la ligne « `\n` »[2]::

f = open()

```
f = open("pays.csv", "r")
pays = [li[:-1].split(',') for li in f]
f.close()
```

with open as f:

```
py
with open("pays.csv", "r") as f:
    pays = [li[:-1].split(',') for li in f]
```

On obtient alors le tableau suivant :

```
>>> pays
[['Pays', 'Capitale', 'Population (ml)'],
 ['France', 'Paris', '68'],
 ['Allemagne', 'Berlin', '82'],
 ['Italie', 'Rome', '60']]
```

Et si on veut supprimer la première ligne si elle contient les titres de colonnes, il suffit d'ajouter `f.readline()` après l'ouverture du fichier :

```
f = open("pays.csv", "r")
f.readline()
pays = [li[:-1].split(',') for li in f]
f.close()   with open("pays.csv", "r") as f:
    f.readline()
    pays = [li[:-1].split(',') for li in f]
```

Exercice corrigé

Importer dans un tableau de tableaux les données du fichier des codes postaux depuis https://www.data.gouv.fr/fr/datasets/base-officielle-des-codes-postaux/.

Réponse

De la même façon qu'on a importer des données en table dans un tableau de tableaux, on peut très bien les importer dans un tableau de p-uplets (pour avoir des données de types différents)[3] ou un tableau de dictionnaires (pour utiliser les descripteurs)[4].

Lire et écrire dans un fichier csv avec le module csv

Le module `csv` permet de lire et écrire facilement des données en table dans un fichier csv.

On commence par créer un objet permettant l'écriture dans le fichier avec la fonction `writer()` du module `csv`. `writerows()` [5] permet ensuite d'écrire un tableau de tableaux directement dans un fichier csv :

```
import csv

pays= [ ['Pays', 'Capitale', 'Population (ml)'],
        ['France', 'Paris', 68],
        ['Allemagne', 'Berlin', 82],
        ['Italie', 'Rome', 60]]

with open('pays.csv', 'w', newline='') as f:
    ecriture = csv.writer(f, delimiter=';')
    ecriture.writerows(pays)
```

Noter qu'à l'ouverture du fichier avec `open()`, il faut indiquer le paramètre `newline=''` sinon des sauts de lignes supplémentaires seront ajoutés entre les lignes.

De la même façon en lecture, il faut créer un objet de lecture avec la fonction `reader()`.

```
import csv

with open('pays.csv', 'r', newline='') as f:
    lecture = csv.reader(f, delimiter=';')
```

`lecture` est un objet qui ne peut pas être affiché directement :

```
>>> lecture
<_csv.reader object at 0x00000224603A0BE0>
```

Mais on peut le parcourir :

```
    for li in lecture:
        print(li)
```

affiche :

```
['Pays', 'Capitale', 'Population (ml)']
['France', 'Paris', '68']
['Allemagne', 'Berlin', '82']
['Italie', 'Rome', '60']
```

Une autre méthode bien utile du module `csv` est la méthode `DictReader` qui prend en argument un ficher csv et le séparateur utilisé et extrait les données dans un tableau de dictionnaires.

```
import csv

with open('pays.csv', 'r', newline='') as f:
    lecture = csv.DictReader(f, delimiter=',')
```

`lecture` est un objet qui ne peut pas être affiché directement :

```
>>> lecture
<csv.DictReader object at 0x00000278966781C0>
```

Mais on peut le parcourir ou le mettre dans un tableau:

```
import csv

with open('pays.csv', 'r', newline='') as f:
    lecture = csv.DictReader(f, delimiter=',')
    print(list(lecture))
```

affiche le tableau de dictionnaire

```
[{'Pays;Capitale;Population (millions)': 'France;Paris;68'},
 {'Pays;Capitale;Population (millions)': 'Allemagne;Berlin;82'},
 {'Pays;Capitale;Population (millions)': 'Italie;Rome;60'}]
```

⚠️ il faut créer le tableau `dico` avant de refermer le fichier !

On trouvera de nombreuses fonctionnalités dans le module `csv`[6]:.

Exercice corrigé

Importer dans un tableau de dictionnaires les données du fichier des codes postaux depuis https://www.data.gouv.fr/fr/datasets/base-officielle-des-codes-postaux/.

Réponse

---

1. Avec le menu Fichier/Ouvrir puis utiliser la fenêtre « Assistant d'importation du texte » pour choisir le séparateur utilisé ou Données/Récupérer des données externes/fichier texte. ↩
2. Ou encore en utilisant la méthode `splitlines()` on peut écrire `>>> pays = [li.split(',') for li in f.read().splitlines()]`. ↩
3. Par exemple avec le code suivant :

```
f = open("pays.csv", "r")
descripteurs = f.readline().split( ',')         # ligne de descripteurs traitée séparément

pays = []
for li in f:
    tab = li[:-1].split(',')
```

```
    puplet = (tab[0], tab[1], int(tab[2]))     # la population est convertie en int
    pays.append(puplet)
f.close()
```

↩

4. Par exemple avec le code suivant :

```
f = open("/pays.csv", "r")
descripteurs = f.readline()[:-1].split( ',')

pays = []
for li in f:
    dico = {descripteurs[0] : li.split(',')[0] ,
            descripteurs[1] : li.split(',')[1],
            descripteurs[2] : int(li.split(',')[2][:-1])
        }
    pays.append(dico)
f.close()
```

↩

5. Ne pas confondre avec `writerow()` qui permet d'écrire ligne par ligne. ↩
6. Voir : https://docs.python.org/fr/3/library/csv.html ↩